

SMART FLOW 2.0

Automated and Remotely Controlled Irrigation System

Bachelor's Thesis

Davit Kavelashvili

2026

Table of Contents

- 1. Introduction**
- 2. Problem Statement**
- 3. Market Research & Competitive Landscape**
- 4. Technical Background**
- 5. System Architecture & Design Decisions**
- 6. Hardware Implementation**
- 7. Firmware Development**
- 8. ESP32 & Cloud Layer Design**
- 9. Testing & Results**
- 10. Economic Analysis**
- 11. Future Work**
- 12. Conclusion**
- 13. References**

1. Introduction

Water is the single most critical resource in agricultural production, yet it remains one of the most inefficiently managed. According to the United Nations Food and Agriculture Organization (FAO), agriculture accounts for approximately 70 percent of all freshwater withdrawals globally, and a significant portion of that water never reaches the root zone of a plant. Traditional irrigation systems — open channels, uncontrolled sprinklers, flood irrigation — operate on fixed schedules that ignore soil moisture levels, weather forecasts, and crop growth stages. The result is a dual failure: crops are either stressed from under-watering or damaged from waterlogging, while limited water reserves are depleted faster than they can be replenished.

In the last few years, cheap microcontrollers and wireless modules have changed what is actually possible for small-scale builders. An STM32 Blue Pill costs about 2 USD. An ESP32 is 4 USD. A TDS probe is 3 USD. The hardware that used to require an industrial controller costing thousands now fits in a small plastic enclosure that can be wired up in a few days. That shift is what makes a project like this possible.

This thesis documents Smart Flow 2.0 — a dual-processor automated irrigation system I designed, built, and tested myself on a 1,500 m² site in Digomi, Tbilisi. The goal was straightforward: build something that a small Georgian farmer can actually afford and use, not another overpriced product designed for commercial operations with big budgets. The result costs around 70 USD in hardware and does more than commercial alternatives charging ten to twenty times that amount.

The core argument is simple: smart irrigation does not have to be expensive. Basically, with the right component choices and a clean architecture, it is possible to build a system that outperforms most commercial products on capability while costing roughly 5 percent of what they charge. This thesis shows that in practice, not just in theory.

1.1 Scope and Objectives

The project covers five main areas:

- **Market Analysis:** Research and documentation of the global smart irrigation market, key competitors, and relevant technical standards.
- **Embedded Firmware:** Design and implementation of a dual-STM32 embedded firmware architecture for sensor acquisition, multi-zone scheduling, and relay-driven valve control.
- **Wireless & Cloud:** Design of the ESP32 cloud bridge layer, including MQTT topic architecture, serial command protocol, and mobile application interface concept.
- **Validation:** Bench testing of sensor accuracy, relay timing, and inter-module communication reliability.
- **Cost Analysis:** Economic analysis comparing the prototype bill-of-materials cost against comparable commercial products.

1.2 Document Structure

Section 2 describes the problem and the proposed solution. Section 3 covers the competitive market. Section 4 gives technical background — sensors, protocols, microcontroller selection. Section 5 covers the system architecture. Sections 6 and 7 are hardware and firmware. Section 8 is the ESP32 and

cloud design. Section 9 is testing and results. Sections 10 to 12 cover economics, future work, and conclusions.

2. Problem Statement

2.1 The Core Problem

The problem is basically three things that reinforce each other, and each one makes the others worse.

Vendor lock-in: Rain Bird, Hunter, and Netafim products are designed to work only with that same manufacturer's hardware. Want to add a sensor? You need their sensor. Need another zone? Their expansion module. Once you choose a vendor, you are stuck with them. For a small farmer who cannot absorb the cost of replacing the whole system every few years, this is a serious and ongoing financial problem.

No remote visibility: Without a way to check the system remotely, you have to be there in person — every day, twice a day in summer. On a site spread across hundreds of meters, that is easily an hour a day for the whole growing season. That is not just a time cost. It means you cannot take a paying job in the city, cannot travel, cannot be anywhere else when the irrigation needs to run. The labor is not optional.

Fixed schedules waste everything: most low-cost irrigation controllers are just timers. Same amount of water, same time every day, regardless of what the soil actually needs. No check for yesterday's rain. No adjustment for the season. FAO research consistently shows that schedule-based drip systems apply 30 to 50 percent more water than crops need. That is not just an environmental problem — at real water and fertilizer costs, it is money being wasted every single day.

2.2 Georgian Agricultural Context

Georgia's agricultural sector is significant — around 7 to 9 percent of GDP, employing roughly 40 percent of the rural workforce — but the technology level is well behind comparable countries. The climate varies enormously across regions: humid subtropical on the Black Sea coast, semi-arid in Kakheti where most wine grapes and many vegetables grow. A single fixed irrigation schedule cannot work across those differences. What is right for Adjara will be too much for plants in a Tbilisi garden, and what works during Tbilisi spring is not enough for Kakheti vines in a dry August.

Kakheti, the main wine-producing region, gets 600 to 700 mm of rain per year — but not when crops need it. The dry stretch from June to August hits exactly during fruit development, when Rkatsiteli and Saperavi vines need consistent moisture for sugar concentration. Without supplemental irrigation during that window, yield and quality both drop. But despite this, precise irrigation at the small-producer level is almost entirely absent in Kakheti. Most vineyards still use flood or furrow irrigation managed purely by eye.

The barrier to adoption of smart irrigation technology in Georgia is not awareness but cost. A basic automated multi-zone controller from Rain Bird or Hunter, imported and installed, represents an investment that exceeds the annual profit margin of a typical small Georgian farm. The import duty structure on electronic components further inflates the final cost. A domestically assembled system using commodity microcontrollers and locally available sensors can bridge this gap at a price point that makes economic sense for even a small producer.

2.3 The Proposed Solution

The system is built around two STM32F103C8T6 microcontrollers and an ESP32 wireless module. The architecture is deliberately split: one microcontroller handles sensing and all irrigation logic, the other handles only relay switching. This separation keeps the firmware on each chip simple and testable, and it means either module can be replaced or upgraded without touching the other.

Modularity Over Monolithism: Rather than a single controller trying to manage all system functions, responsibilities are distributed. A dedicated Sensor Module handles all data acquisition and irrigation scheduling logic. A dedicated Relay Module handles all physical actuator switching. The separation means that either module can be replaced, upgraded, or replicated independently without affecting the other. Adding zones means adding another Relay Module; adding sensors means extending the Sensor Module firmware.

Openness Over Lock-in: All communication between modules uses standard UART serial protocol with a human-readable ASCII command set. All cloud communication uses MQTT over standard TCP/IP. Any mobile device, any cloud provider, and any custom application can integrate with the system without proprietary drivers or licensing agreements.

Affordability Without Sacrifice: The entire hardware bill of materials — two STM32 development boards, one ESP32 module, relay boards, sensors, power supply components, and an enclosure — is achievable for under 50 USD at current component prices. This is a cost level that is genuinely accessible to small Georgian and regional agricultural producers.

3. Market Research & Competitive Landscape

3.1 Global Market Overview

The global smart irrigation market was valued at approximately 2.14 to 2.54 billion USD in 2024 and is projected to reach 4.15 billion USD by 2030 and potentially 8.44 billion USD by 2034. This trajectory represents a compound annual growth rate (CAGR) of 14.1 to 14.28 percent — significantly outpacing the growth rate of the overall agricultural technology sector, which averages 12 percent annually.

The primary catalyst for this expansion is the global response to water scarcity driven by climate change. Nearly 50 percent of water applied through traditional irrigation systems is lost to evaporation, runoff, and deep percolation before reaching plant root zones. As water costs rise and regulatory pressure on agricultural water use increases in Europe, North America, and parts of Asia, the economic case for precision irrigation strengthens substantially.

North America currently represents the largest regional market, driven by large-scale commercial farming operations, early adoption of precision agriculture, and strong venture capital investment in AgTech. However, the Asia-Pacific region is identified by market analysts as the segment with the highest future growth potential, driven by rapid urbanization, the development of smart city infrastructure, and urgent government programs to address freshwater scarcity in agricultural zones.

3.2 Key Competitors and Product Analysis

There are three categories of products competing in this space: global enterprise players, mid-market IoT vendors, and open-platform alternatives. Each tier targets a different buyer and has different trade-offs.

Tier 1 — Enterprise Systems

Rain Bird (USA) is the biggest name in large-scale commercial irrigation. Their IQ4 platform offers cloud connectivity, modular zone expansion to hundreds of zones, and API integration with work-order software. It is designed for golf courses, municipal installations, and large commercial farms. Entry-level cost starts at 3,000 USD and goes up from there. For a small Georgian farmer, Rain Bird's intelligent platform is not an option — it never even appears in the same search.

Netafim (Israel) is the pioneer of drip irrigation and what they term 'nutrigation' — the simultaneous delivery of water and dissolved nutrients through the drip lines themselves. Their NetBeat precision platform provides field-level monitoring with soil sensors, satellite imagery integration, and AI-driven scheduling recommendations. Netafim systems are designed primarily for large-scale vegetable and fruit production in regions where water is critically scarce, such as Israel, Spain, and Australia. Their pricing model is subscription-based and project-specific, making cost comparison difficult, but typical large farm deployments are in the range of tens of thousands of USD.

Toro (USA) has moved toward the intersection of irrigation and autonomous equipment, following their acquisition of Left Hand Robotics. Their Lynx Smart System targets centralized management for commercial turf and commercial-scale operations. Their recent focus is data analytics and predictive maintenance, not basic zone control.

Tier 2 — Mid-Market IoT Controllers

Rachio (USA) is the best-known consumer-facing smart irrigation brand. Their Rachio 3 offers 8 or 16 zones, WiFi, Alexa and Google Home integration, and automatic weather-based schedule adjustment via the American Weather Intelligence database. The 8-zone version retails for 170 to 230 USD. The problem for agricultural use is clear: it is built for residential garden beds, has no TDS or EC sensor support, no fertigation, and the smart features require an ongoing internet subscription. It works well for what it is. What it is does not include anything a real farm needs.

Galcon (Israel) produces a broader range of controllers that span residential, commercial, and light agricultural applications. Their 7001BTD Bluetooth controller series and the 9001DC battery-operated valve controller are widely used for small farms and greenhouse operations. Galcon products are more affordable than Rain Bird or Netafim, with individual zone controllers available from 80 to 300 USD, but they share the same fundamental limitation: no integrated water quality sensing and no fertigation management.

Hunter Industries (USA) makes controllers and sensor technology for commercial and large residential installations. Their HC-1200C supports up to 48 zones and integrates with the Hunter Predictive Watering (WAND) system for weather-adjusted scheduling. Hunter also makes the Soil-Clik moisture sensor, which can pause irrigation when soil is already wet. The full Hunter ecosystem is proprietary — same lock-in problem as Rain Bird.

Tier 3 — Open Platform Alternatives

OpenSprinkler is the most prominent open-source hardware irrigation controller. Built around an ATmega with an ESP8266 or Raspberry Pi companion, it supports up to 72 cascaded zones, has a full API, integrates with weather services, and costs around 60 to 80 USD as a kit. OpenSprinkler proves the open-platform concept works and that users will self-assemble — but it has no TDS, pH, or pressure sensor inputs, and no fertigation mechanism. It shows the market gap exists. It does not close it for agricultural applications.

3.3 Competitive Comparison Table

The following table summarizes the key feature and cost comparison between Smart Flow 2.0 and the most relevant competitive products:

Feature	Smart Flow 2.0	Rain Bird IQ4	Rachio 3	Galcon 9001	OpenSprinkler
Max Zones	11 (expandable)	Unlimited	8 / 16	1–4	72
TDS Sensing	Yes	No	No	No	No
pH Sensing	Yes	No	No	No	No
Pressure Monitoring	Yes	No	No	No	No
Fertigation Control	Yes (servo mixer)	No	No	No	No
Remote Control	MQTT / App	Cloud (paid)	WiFi App	Bluetooth	WiFi API

Open Protocol	Yes (UART/MQTT)	No	Partial	No	Yes
Approx. Cost (USD)	~\$45	\$3,000+	\$170–230	\$80–150	\$60–80
Agricultural Focus	Yes	Commercial	Residential	Light Agri	Residential

The comparison table makes it clear: Smart Flow 2.0 is in a category by itself. No product at a comparable price point combines live water quality monitoring, automated fertigation, multi-zone relay control, and an open remote-access protocol. OpenSprinkler is the closest on openness but has no sensor inputs. Netafim and Rain Bird have the agricultural functionality but at prices that exclude 99 percent of small Georgian farms. Smart Flow 2.0 is the only thing in the gap.

3.4 Identified Market Gap

Based on this competitive analysis, the unaddressed market segment is clearly defined: small to medium agricultural operations (1 to 10 hectares) that require multi-zone irrigation with basic water quality feedback and remote management capability, at a total system cost below 200 USD. This segment includes the majority of Georgian farming operations, small Central Asian and Eastern European cooperative farms, greenhouse operators, and urban farming installations. No existing commercial product serves this segment effectively. Smart Flow 2.0 is designed specifically to address it.

3.5 Methodology

This project followed an applied engineering research methodology structured across five sequential phases: problem investigation, hardware selection, iterative prototyping, field deployment, and ongoing validation. Real-world feedback from the live deployment site shaped decisions at every stage of development.

Phase 1 — Problem Investigation

The project began with a review of commercial irrigation products and published research on precision agriculture and water management. The goal was to identify a concrete unmet need rather than replicate an existing solution. This phase produced the competitive gap analysis in Section 3: no affordable product below \$200 combines multi-zone control, water quality monitoring, and remote access in a single open system.

Phase 2 — Hardware Selection

Component selection was governed by three hard constraints: total cost under \$100 USD, availability through standard consumer channels, and a feature set sufficient to support the intended functionality. Each component was evaluated against alternatives — STM32F103 over Arduino, ESP32 over ESP8266, analog TDS probes over digital soil moisture sensors — before being finalised.

Phase 3 — Iterative Prototyping

Firmware was built incrementally: the Relay Module first as the simplest subsystem, then the Sensor Module ADC pipeline, then the inter-module UART protocol. Each layer was validated with a serial terminal before the next was built on top. This isolated faults at each integration step rather than requiring full-system debugging from the start.

Phase 4 — Field Deployment

The complete system was installed at a private 1,500 m² site in Digomi, Tbilisi. The site was chosen because it matched the target use case exactly: new plantings with sensitive watering requirements, four physically separate irrigation zones, and a municipal water supply with known intermittent pressure problems. A 1,000-litre reservoir was integrated to buffer supply interruptions.

Phase 5 — Ongoing Validation

The system has operated continuously since deployment. Validation is conducted through direct observation of plant health, periodic sensor calibration checks, and comparison of readings against reference solutions. The deployment remains active at the time of writing — providing a live performance record rather than a short-duration bench test.

3.6 Target User Profile

Smart Flow 2.0 was designed for a specific but underserved category of landowner: the small-scale horticultural or agricultural site manager in Georgia and comparable developing-market contexts. This is not the large commercial farm — those operators already have enterprise irrigation platforms. The target user manages a plot of roughly 500 to 3,000 square metres: large enough to justify automation, too small to absorb the cost of commercial solutions.

User Characteristics

The typical user has no formal training in agronomy or embedded systems. They know that plants need water and that incorrect watering causes damage, but they have no tools to quantify what 'correct' looks like in real time. Their site often contains a mix of vegetables, fruit trees, ornamental shrubs, and perennials across zones with meaningfully different water and nutrient needs.

Newly established plantings are the highest-risk category. Trees, shrubs, and seedlings installed within the past one to two growing seasons have immature root systems that are intolerant of both drought stress and waterlogging. For this user, losing new plants to an irrigation failure costs more — in money, time, and replanting effort — than the control system itself.

Key Pain Points

- **1. Time cost of manual irrigation — visiting the site twice daily to water by hand.**
- **2. No visibility into water quality — nutrient levels and pH are invisible without sensors.**
- **3. No remote access — any absence means missed irrigation or arranging a caretaker.**
- **4. Municipal supply interruptions — no buffer means plant stress during pressure drops.**
- **5. No zone-specific control — a single hose treats all zones the same regardless of need.**

3.7 Real-World Deployment: Digomi Site, Tbilisi

Smart Flow 2.0 was not developed as a theoretical exercise. The complete system was built, installed, and commissioned at a private horticultural site in the Digomi district of Tbilisi, Georgia. This section documents the site conditions, the situation before installation, and the operational change that followed.

Site Description

The site covers approximately 1,500 square metres and contains a mix of newly planted vegetable beds, ornamental shrubs, and young trees positioned along the perimeter walls. All plantings were newly established at the time of installation, making consistent and calibrated irrigation directly critical to plant survival. The site was divided into four irrigation zones:

- Zone 1 — Large section: mixed vegetables and shrubs. High water demand, irrigated daily.
- Zone 2 — Large section: vegetables and ornamental plantings. Medium demand.
- Zone 3 — Large section: mixed shrubs and ground cover. Medium demand, less frequent cycles.
- Zone 4 — Wall section: young trees along the perimeter. Lower demand, drip-feed only.

Water Supply

The site connects to the Tbilisi municipal water supply, which experiences periodic low-pressure periods during summer peak demand. A 1,000-litre holding reservoir was installed upstream of the main pump. The pump draws from the reservoir rather than directly from the mains, providing a buffer that sustains approximately 45 to 60 minutes of full irrigation when municipal pressure is insufficient. The reservoir refills automatically from the mains when pressure is restored.

Before Installation

Prior to Smart Flow 2.0, the entire 1,500 m² site was irrigated manually by hand using a garden hose — two daily visits of 30 to 40 minutes each during peak summer conditions. Water was applied by visual inspection with no measurement or scheduling. There was no way to detect nutrient deficiencies, no zone-specific control, and no remote access. Any absence from the site meant either stopping irrigation entirely or arranging a third party to do it manually.

After Installation

Irrigation now runs fully automatically across all four zones. The Sensor Module monitors TDS and pressure continuously, adjusting fertilizer concentration via the stepper-controlled mixing valve to

maintain the configured nutrient target. Each zone runs on an independently configured duration within the 13-state irrigation sequence, repeating on schedule without any manual input.

Manual site visits for irrigation have been eliminated entirely. The 1,000-litre reservoir has bridged multiple municipal supply pressure drops since installation. Sensor readings are accessible remotely via the MQTT interface. Plant health across all four zones is consistent with expected development for the age and species of the plantings.

Current Status

The system remains in active operation at the time of writing. This makes the Digomi installation a live, ongoing performance record — validating not just that the system can be built, but that it sustains continuous operation under real field conditions in the Georgian climate.

3.8 Climate Change and the Water Crisis in Georgia

The inefficiency of current irrigation practice in Georgia is not a static problem. The underlying conditions — water availability, temperature, rainfall reliability — are deteriorating, and the rate of deterioration is accelerating. Understanding the climate context is essential for understanding why smart irrigation technology is not a luxury upgrade for Georgian farmers but an increasingly urgent necessity.

Georgia's mean annual temperature has increased by approximately 1.1°C since 1961, with the warming trend accelerating significantly in the past two decades. Eastern Georgia — the Kakheti region, Georgia's most agriculturally productive territory — has been disproportionately affected. Summer temperatures in Kakheti regularly exceed 38°C and have reached 42°C in recent extreme events. At these temperatures, the evapotranspiration rate of crops — the rate at which plants and soil lose water to the atmosphere — increases dramatically, raising irrigation demand precisely when water supply is under the most stress.

Rainfall patterns across Georgia are becoming less predictable and more variable. The total annual precipitation in Kakheti (600 to 700 mm) has not changed dramatically in aggregate, but its distribution throughout the year has shifted. Rain that was historically spread across the growing season is increasingly concentrated in short intense events separated by longer dry periods. For farmers operating on fixed irrigation schedules — or no schedule at all — this variability is deeply damaging: plants experience waterlogging during the intense events and drought stress during the gaps, with neither condition detected until visible damage appears.

The Caucasus mountain range, which provides the snowmelt and glacial water that feeds Georgia's major rivers — including the Mtkvari (Kura), on which eastern Georgian agriculture depends — is

experiencing rapid glacial retreat. Studies published by the Georgian National Environmental Agency estimate that Caucasus glacier volume has reduced by approximately 20 percent since 1960, with the rate of retreat increasing since the 1990s. Reduced glacial storage means lower river flows during the critical summer irrigation season, creating increasing competition between agricultural, municipal, and industrial water users.

The paradox facing Georgian agriculture is sharper than in most other contexts: climate change is simultaneously increasing the amount of water that crops need and reducing the reliability of the supply. Farmers are being squeezed from both sides. Fixed-schedule irrigation systems, already wasteful under stable conditions, become significantly more damaging under these volatile conditions: they apply the same amount of water regardless of whether the crop needs it, regardless of what happened in the past 24 hours, and regardless of what the pressure in the supply pipe is.

Sensor-driven irrigation — the approach at the core of Smart Flow 2.0 — directly addresses both sides of this squeeze. TDS monitoring detects the nutrient concentration in the water supply in real time, compensating for the seasonal variation in river and reservoir chemistry that accompanies variable rainfall. Pressure monitoring detects supply shortfalls before they translate into plant stress. And a configurable zone-by-zone schedule allows irrigation to be adapted to changing conditions without requiring the farmer to be physically present to make the adjustment.

The climate trajectory for Georgia points in one direction. The problem of water management in agriculture will not become easier in the coming decades. The question is whether the tools available to Georgian farmers will advance fast enough to keep pace with it.

3.9 Social Impact on Rural Farmers

The efficiency argument for smart irrigation — less water, less fertilizer, better yields — is well documented and straightforward to quantify. The social argument is harder to measure but arguably more important for understanding why this project matters beyond engineering.

Georgia's rural population is aging and shrinking. According to Geostat data, the proportion of the agricultural workforce under 35 fell by more than 30 percent between 2010 and 2022. Young people are leaving villages for Tbilisi and, increasingly, for Europe. The reasons are economic — urban wages are higher — but they are also qualitative. Agricultural work as currently practiced is physically demanding, time-consuming, and offers almost no flexibility. A farmer who must be present at their land twice a day every day during the growing season cannot hold a second job, cannot travel, and cannot build a life that resembles what their urban contemporaries experience.

Manual irrigation is one of the most time-consuming of these daily obligations. On a 1,500 m² site — the scale of the Digomi deployment — hand watering with a hose requires approximately 60 to 80 minutes of physical presence every day during the summer months. That is not just time spent watering. It is time that cannot be used for anything else. It is the reason a farmer cannot take a weekend trip, accept a day of paid work in the city, or simply rest.

Automating irrigation does not just save water. It returns time. And for a Georgian small farm operator who is managing land alongside other economic pressures, the return of that time is not a minor convenience — it can be the difference between farming being viable as a way of life or not.

There is also a generational dimension. The land management traditions of Georgian family farming — particularly in wine-producing regions like Kakheti — represent cultural and economic heritage that spans centuries. The decline of small-scale farming is not just an agricultural statistic; it represents the erosion of a way of life that has shaped Georgian culture, cuisine, and identity. If automation makes small-scale farming economically competitive and personally sustainable for a younger generation, it becomes a tool for cultural continuity, not just crop production.

Smart Flow 2.0 does not solve the structural economic challenges facing Georgian agriculture. But it addresses one of the most tangible daily burdens — the obligation of manual presence — in a way that is accessible at the scale and budget of the farmers who need it most.

3.10 Adoption Challenges and Human Factors

One of the most persistent findings in agricultural technology research is that the gap between what technology can do and what farmers actually adopt is rarely explained by the technology itself. In Georgia, as in most developing agricultural contexts, the barrier to adopting smart irrigation is not a lack of awareness that better solutions exist. Farmers know their current approach is inefficient. The barrier is a combination of cost sensitivity, justified distrust of complexity, and the absence of local support infrastructure.

Previous generations of 'smart' irrigation products that reached Georgian distribution channels failed predictably for the same set of reasons. They were expensive. Their setup required technical knowledge that most farmers do not have. Their documentation was in English or Russian. They depended on stable internet connectivity for basic functions — connectivity that remains unreliable in most rural Georgian villages. And crucially, when they broke, there was no one locally trained to fix them. A \$400 controller that stops working after one season and cannot be repaired is not a solution; it is a risk.

The APMA 2021 survey finding that 71 percent of Georgian farmers cite initial capital cost as the primary barrier to pressurized irrigation adoption is frequently interpreted as a simple price problem. In practice it is a risk-management problem. A farmer managing 1.5 hectares of vegetables on an annual income of approximately 4,000 to 6,000 USD cannot absorb the financial shock of a \$500 system failure. At that income level, every equipment purchase is evaluated not just by its sticker price but by its failure cost.

Smart Flow 2.0 was designed with these adoption realities as first-class constraints:

- Failure cost is manageable. The total component cost of approximately \$70 means that even a complete system failure — every component destroyed — is a loss that can be recovered from within a single season. Compare this to a commercial alternative where a single replacement controller costs more than the entire Smart Flow 2.0 bill of materials.

- Local autonomy. The core irrigation logic runs entirely on the STM32 microcontrollers without any dependency on WiFi, cloud services, or the internet. A farmer in a village with no mobile data signal can still run a fully automated irrigation schedule. The cloud layer adds convenience; it is not required for basic operation.
- Repairability with standard parts. Every component in Smart Flow 2.0 — the microcontroller boards, relay modules, sensors — is a commodity item available from electronics distributors globally and through online channels shipping to Georgia within two weeks. There is no proprietary hardware that requires manufacturer authorisation to replace.
- Visible proof rather than marketing material. The most powerful adoption tool for any agricultural technology in Georgia is a working example that a farmer can visit and observe. The Digomi installation serves precisely this purpose: a live system that anyone can see running on a real site, managing real plants, in Georgian summer conditions.

The human factors question in agricultural technology adoption is ultimately about trust. Farmers trust what they can see, understand, and fix themselves. Smart Flow 2.0's design philosophy — open protocols, commodity hardware, local autonomy — is as much an adoption strategy as it is an engineering choice.

4. Technical Background

4.1 Irrigation Fundamentals

Precision irrigation is a simple idea with complicated implementation: give each plant the water it actually needs, when it needs it, and nothing more. In practice this means understanding the relationship between water delivery, plant uptake, and soil retention — and having sensors and actuators that can act on that understanding in real time.

Drip irrigation — the delivery mechanism used in this project — delivers water directly to the root zone of each plant through a network of emitters operating at low pressure (0.5 to 2.0 bar). Compared to sprinkler irrigation, drip systems reduce evaporation losses by 30 to 50 percent and enable fertigation — the delivery of dissolved nutrients through the water supply — with precise concentration control. The physics of drip emitter design involves pressure compensation: quality emitters maintain constant flow rate across a range of input pressures, which is critical for uniform water distribution across uneven terrain.

Fertigation, the simultaneous delivery of irrigation water and plant nutrients, is the most technically demanding aspect of precision irrigation. Professional fertigation systems use EC (electrical conductivity) as a proxy for total dissolved nutrient concentration. Since EC is essentially TDS measured in milliSiemens per centimeter (mS/cm) rather than parts per million (ppm), a TDS sensor in the delivery line provides a real-time feedback signal for fertilizer concentration control. The typical target EC for vegetable crops is 2.0 to 3.5 mS/cm during vegetative growth and 3.5 to 5.0 mS/cm during fruit development.

4.2 Water Quality Parameters

Total Dissolved Solids (TDS) and Electrical Conductivity (EC)

TDS — Total Dissolved Solids — is the total mass of dissolved material in water, expressed in ppm (parts per million) or mg/L. In irrigation practice, TDS is used as a quick measure of fertilizer concentration in the delivery water. The approximate relationship between TDS and EC (electrical conductivity) is: $EC \text{ (mS/cm)} \approx TDS \text{ (ppm)} / 640$, though the exact factor depends on what ions are dissolved.

Conductivity-based TDS sensors — the type used in this project — measure TDS indirectly by applying an alternating current between two electrodes immersed in the solution and measuring the resulting conductance. The alternating current prevents electrolytic polarization of the electrodes, which would cause measurement drift over time. Temperature compensation is applied to the raw conductance reading because ionic mobility increases with temperature at approximately 2 percent per degree Celsius — a significant correction at the scale of agricultural applications where solution temperature can range from 10°C to 35°C across seasons.

pH

pH measures the hydrogen ion concentration in a solution on a logarithmic scale from 0 (strongly acidic) to 14 (strongly alkaline), with 7 representing neutral. For agricultural irrigation, the optimal pH range for most vegetable crops is 5.8 to 6.5. Outside this range, nutrient availability is reduced even when the correct quantities of fertilizer are present — a condition called nutrient lockout. Iron, manganese, and zinc become unavailable above pH 7.0. Phosphorus availability peaks at pH 6.5 and drops sharply on either side. Calcium and magnesium availability decreases below pH 5.5.

pH electrodes operate on the Nernst equation: the potential difference generated across a glass membrane between the sample solution and an internal reference solution is proportional to the pH difference. The output voltage of a typical analog pH electrode is approximately 59.16 mV per pH unit at 25°C, with a midpoint (pH 7) corresponding to approximately 2.5V on a 5V reference. The analog signal is read by the STM32 ADC and converted to pH using a linear calibration: $pH = -5.70 \times (ADC_voltage / reference) + calibration_offset$, where the calibration offset is determined empirically using buffer solutions of known pH.

Pressure

Line pressure monitoring serves two purposes in an irrigation system: detecting pump failures (pressure below threshold indicates pump not running or pipe break) and preventing over-pressure damage to drip emitters and solenoid valves. The pressure sensor used in this project is a resistive pressure transducer with a 0 to 10 bar measurement range and an analog voltage output of 0.5 to 4.5V. The conversion from ADC reading to pressure in bar is linear: $P = (V_out / V_supply - 0.1) \times (P_max / 0.8)$, where V_supply is the reference voltage and P_max is the sensor's full-scale range.

4.3 Microcontroller Selection

The STM32F103C8T6 — the 'Blue Pill' — was chosen for both microcontroller roles. The reasons were practical: it is cheap (around 2 USD), it has two hardware UARTs (critical for running sensor-to-

relay and sensor-to-ESP32 communication simultaneously), and its 12-bit ADC gives better resolution than the 10-bit ADC on an Arduino. The STM32CubeIDE with CubeMX also generates the peripheral initialization code automatically, which saved significant time.

The STM32F103 is a 32-bit ARM Cortex-M3 processor running at up to 72 MHz, with 64 KB of flash memory and 20 KB of SRAM. It provides three USARTs, two SPI interfaces, two I2C interfaces, and a 12-bit ADC with 10 external channels. For the purposes of this project, the critical peripherals are the two UART ports on the Sensor Module (one for relay communication, one for ESP32 communication) and the ADC for sensor reading. The 12-bit ADC provides 4096 discrete levels over the reference voltage range, giving a resolution of approximately 0.8 mV per bit at 3.3V — sufficient for the precision required by the TDS and pH measurements.

The Arduino was considered and rejected. The ATmega328P is an 8-bit processor with a single hardware UART and a 10-bit ADC. The TDS formula uses cubic arithmetic — 8-bit floating point introduces rounding errors that matter at this precision level. And with only one UART, running serial to both the relay module and the ESP32 simultaneously requires software serial, which is unreliable at higher baud rates. The STM32 solves both problems with headroom to spare.

The ESP32-WROOM-32 was selected as the wireless gateway module rather than integrating WiFi directly into the STM32 firmware. The ESP32 is a dual-core Xtensa LX6 processor with integrated 2.4 GHz WiFi (802.11 b/g/n) and Bluetooth 4.2. Running WiFi and MQTT protocol stacks simultaneously on an STM32 without an external module would require an external WiFi chip (such as the ESP8266 or WINC1500), adding complexity and a second programming environment. By using the ESP32 as a dedicated wireless bridge with its own firmware, the STM32 firmware remains focused exclusively on real-time control logic, and the ESP32 firmware focuses exclusively on connectivity — a clean separation of concerns.

4.4 Communication Protocols

UART (Universal Asynchronous Receiver/Transmitter)

UART is the oldest and most widely used serial communication protocol in embedded systems. It transmits data asynchronously: both sender and receiver must agree on the baud rate (bits per second) in advance, and each byte is framed with a start bit, 8 data bits, and one or two stop bits. In this project, all UART interfaces operate at 9600 baud, which provides a byte transmission time of approximately 1.04 ms — fast enough for the system's command throughput requirements while remaining reliable over the unshielded wiring runs expected in a field installation.

MQTT (Message Queuing Telemetry Transport)

MQTT is a lightweight publish-subscribe messaging protocol designed specifically for constrained devices and unreliable network conditions, making it the de facto standard for IoT applications. Originally developed by IBM in 1999 for monitoring oil pipelines via satellite links, MQTT was standardized by OASIS in 2014 (MQTT 3.1.1) and updated in 2019 (MQTT 5.0).

The protocol operates on a broker-based architecture: devices do not communicate directly with each other but instead publish messages to named topics on a central broker. Other devices subscribe

to those topics and receive messages when they are published. This decoupling is architecturally significant — the mobile app and the irrigation controller never need a direct connection; they simply publish and subscribe to the same broker, which can be hosted in the cloud at negligible cost.

MQTT defines three quality-of-service levels: QoS 0 (at most once, fire and forget), QoS 1 (at least once, with acknowledgment), and QoS 2 (exactly once, with four-way handshake). For irrigation control commands — where duplicate execution of a 'start irrigation' command could cause unintended watering cycles — QoS 1 is the appropriate choice. For sensor telemetry where occasional message loss is acceptable, QoS 0 reduces overhead and improves throughput.

The designed MQTT topic structure for Smart Flow 2.0 is: smartflow/sensors for outbound sensor data (TDS, pH, pressure published as JSON), smartflow/cmd for inbound control commands (start irrigation, set zone durations, set TDS target), and smartflow/status for system heartbeat and fault reporting. This structure is clean, easy to monitor with standard MQTT tooling, and supports multiple independent installations by adding a device ID prefix if needed.

4.5 Relay Control Principles

A relay is an electrically operated switch. The STM32 GPIO pins output 3.3V logic signals, which cannot directly switch the 12V solenoid valves and AC pumps used in the irrigation system. A relay module interposes a low-voltage control circuit (driven by the STM32) and a high-voltage load circuit (driving the solenoid or pump). The relay coil typically requires 5V and 50 to 80 mA to actuate — within the capability of a GPIO pin when driven through a transistor, or directly managed by an optocoupler-isolated relay driver board as used in this project.

Solenoid valves are electromechanical devices that convert an electrical input into a mechanical valve opening or closing action. The valve body contains a coil that, when energized, generates a magnetic field that moves a ferromagnetic plunger against a spring, opening or closing the flow orifice. For irrigation applications, normally-closed (NC) solenoid valves are preferred: in the unpowered state, the valve is closed, which means a power failure results in the irrigation stopping rather than running uncontrolled.

4.6 Stepper Motor for Fertilizer Dosing

The fertilizer concentration control mechanism in Smart Flow 2.0 uses a stepper motor to position a mixing valve that blends concentrated fertilizer solution with the irrigation water supply. A stepper motor divides a full rotation into a fixed number of discrete steps — in this project, the motor has a step angle of 1.8 degrees (200 steps per revolution), providing fine-grained positional control without requiring a position feedback sensor.

The motor is driven in a simplified two-wire mode: one GPIO pin drives the motor in the decrease-concentration direction, one drives it in the increase-concentration direction, with a 100 ms pulse width per step. The firmware maintains a position counter (0 to 90 representing the valve's physical travel range) so that the controller always knows the current valve position and can prevent over-travel in either direction. This position is used by the TDS feedback loop: if measured TDS exceeds the target, the valve is stepped toward dilution; if TDS is below target, the valve steps toward higher concentration.

5. System Architecture & Design Decisions

5.1 Overall System Block Diagram

The system has four layers stacked on top of each other: sensing and scheduling (Sensor Module), physical actuation (Relay Module), cloud connectivity (ESP32), and user interface (mobile app and cloud broker). Each layer only knows about the layer directly below it — the mobile app talks to the cloud, the cloud talks to the ESP32, the ESP32 talks to the Sensor Module, and the Sensor Module talks to the Relay Module. This separation means each piece can be changed or upgraded without rewriting everything else.

- **User → Cloud:** The mobile app publishes control commands (zone selection, irrigation durations, TDS target) to the MQTT broker on the smartflow/cmd topic.
- **Cloud → ESP32 → Sensor Module:** The ESP32 subscribes to smartflow/cmd, receives the commands, translates them into the STM32 serial protocol, and transmits them over UART to the Sensor Module.
- **Sensor Module → Relay Module:** The Sensor Module processes the commands, reads the sensors, manages the irrigation sequence timer, and transmits relay control commands over UART to the Relay Module.
- **Relay Module → Physical Actuators:** The Relay Module receives relay commands and switches the corresponding GPIO outputs, which drive the relay boards connected to solenoid valves, pumps, and the stepper motor.
- **Sensor Module → ESP32 → Cloud:** The Sensor Module periodically transmits current TDS, pH, and pressure readings back over UART to the ESP32, which publishes them to the MQTT broker on the smartflow/sensors topic.
- **Cloud → User:** The mobile app subscribes to smartflow/sensors and displays live readings to the user.

5.2 Rationale for Dual-STM32 Architecture

Using two separate STM32 chips instead of one was a decision driven entirely by installation reality, not theory. In a real field installation, the sensors sit at the central manifold near the water source and fertilizer tank. The relay board that switches zone valves has to be where the valve wiring converges — which can be tens of meters away. Running 11 relay control wires across a field is impractical. Running a single two-wire UART cable to a remote relay node is straightforward. The split architecture came directly from that physical constraint.

Running 11 relay control wires from a central controller to valves spread across a field is impractical. Running a single UART pair (two wires at 5V logic) from the Sensor Module to a local Relay Module near the valve bank is both simpler and more reliable. The Relay Module becomes a remote actuator node — it receives structured commands and executes them locally, without any application logic of its own. This architecture is extensible: a second Relay Module could be added for a second valve bank by daisy-chaining the UART line or adding a second UART port to the Sensor Module.

5.3 Communication Protocol Design

The inter-module communication protocol between the Sensor Module and the Relay Module uses a three-digit ASCII command string followed by a carriage return and newline (`\r\n`). The command format is: `[1][2][3]\r\n`, where the first digit indicates the action (0 = off, 1 = on) and digits 2 and 3 identify the relay channel (01 to 11). Additional commands M0 and M1 control the stepper motor direction. This protocol is intentionally human-readable: commands can be monitored and debugged with a standard serial terminal without any decoding tool.

The ESP32 to Sensor Module protocol uses a richer command set: numeric codes for requests (999 = request sensor data, 801 = start irrigation sequence), alphanumeric codes for configuration (S1xxxx through S4xxxx for zone irrigation durations, Exxxx for TDS target, Rxx for zone relay selection). The ESP32 effectively translates between the structured MQTT JSON commands from the cloud and the compact ASCII serial commands that the STM32 can efficiently parse.

5.4 Irrigation Sequence State Machine

The irrigation process is managed as a 13-state finite state machine driven by the 1-second hardware timer interrupt. The state machine is designed to sequence through up to four irrigation zones, each with its own configurable duration, while managing the main pump, the fertilizer pump, and the TDS feedback loop independently. The state transitions are:

State 0 (Idle): No irrigation in progress. The system monitors sensors and responds to commands. State 1 (Pump Start): Main pump is turned on. The first selected zone valve is opened. A 5-second stabilization delay is initiated to allow line pressure to build before opening the fertilizer circuit. State 2 (First Zone Active): After the 5-second stabilization, the fertilizer pump is activated. The irrigation duration counter begins counting down from interval1 (the first zone's configured duration). State 3 (First Zone TDS Control): While the first zone is irrigating, the TDS feedback loop is active — if TDS exceeds 1.3 mS/cm, the mixer valve is stepped toward dilution; if TDS falls below 1.0 mS/cm, the valve is stepped toward concentration. When the interval elapses, the fertilizer pump is switched off. States 4–12 repeat this pattern for zones 2 through 4. State 13 (Completion): The last zone valve is closed and the main pump is switched off. The system returns to State 0.

5.5 Power System Design

The power architecture supports both grid-connected and off-grid operation. In grid-connected mode, a 12V DC power adapter provides the main supply. An LM2596 buck converter steps this down to 5V for the STM32 Blue Pill boards and relay driver modules, and a second LM2596 or LDO regulator provides 3.3V for sensor reference voltage. In off-grid mode, a 20W solar panel charges a lithium-ion battery pack through a charge controller, providing the same 12V bus from stored energy.

The total current draw of the system in active irrigation mode is approximately 400 to 600 mA at 12V: two STM32 boards draw approximately 30 mA each, the ESP32 draws up to 240 mA during WiFi transmission bursts, the relay modules draw approximately 60 to 80 mA per active relay coil, and the solenoid valves draw approximately 200 to 300 mA each. A 10 Ah lithium-ion battery can support approximately 10 to 15 hours of continuous operation, which is sufficient for a full daily irrigation cycle.

6. Hardware Implementation

6.1 Sensor Module Hardware

The Sensor Module is built around an STM32F103C8T6 development board (Blue Pill form factor) running at 72 MHz from an 8 MHz external crystal oscillator with the internal PLL configured for $\times 9$ multiplication. The crystal oscillator is used in preference to the internal RC oscillator because the internal oscillator's ± 1 percent frequency tolerance is insufficient for reliable 9600 baud UART communication over extended periods — accumulated timing error causes UART framing errors.

The TDS sensor probe connects to ADC channel 0 (PA0). The probe outputs an analog voltage proportional to the electrical conductivity of the irrigation water, which is then digitized by the 12-bit ADC. A rolling buffer of 30 ADC samples is maintained in firmware, and the average is used to compute TDS through the cubic polynomial calibration formula derived from laboratory measurements with known-concentration calibration solutions.

The pH electrode connects to ADC channel 1 (PA1). The raw ADC reading is converted to pH using the linear calibration formula: $\text{pH} = -5.70 \times (V_{\text{ADC}} \times 5.0 / 4096) / 6 + \text{calibration_offset}$, where the calibration_offset of 27.34 was determined by two-point calibration with pH 4.01 and pH 6.86 buffer solutions. A 10-sample moving average is applied to the pH ADC readings to reduce noise from the high-impedance electrode signal path.

USART1 (PA9/PA10) communicates with the Relay Module at 9600 baud. The Sensor Module is the master in this relationship: it only transmits commands and does not expect responses from the Relay Module. USART3 (PB10/PB11) communicates with the ESP32 at 9600 baud in full duplex mode, receiving configuration commands and forwarding sensor data on request.

6.2 Relay Module Hardware

The Relay Module is built around a second STM32F103C8T6 board running at 8 MHz from the internal RC oscillator. The lower clock speed is intentional — the Relay Module performs no computationally intensive tasks; its entire function is to receive UART commands and toggle GPIO outputs. Running at 8 MHz reduces power consumption compared to the 72 MHz Sensor Module.

The GPIO outputs drive an optocoupler-isolated 8-channel relay board plus additional discrete relay modules for the remaining channels. The optocoupler isolation is important for field reliability: the inductive kickback from solenoid valve switching can generate voltage spikes of several hundred volts on the load side of the relay. The optocoupler ensures these spikes cannot propagate back into the STM32 GPIO inputs, which would cause unpredictable resets or permanent GPIO damage.

The relay output pin assignments are:

- PA2 → Relay 1: Main circulation pump (12V DC, 300W)
- PB5 → Relay 2: Fertilizer dosing pump (12V DC, 5W)
- PA3 → Relay 3: Zone 1 solenoid valve
- PA4 → Relay 4: Zone 2 solenoid valve
- PB4 → Relay 5: Zone 3 solenoid valve

- PB3 → Relay 6: Zone 4 solenoid valve
- PA15 → Relay 7: Zone 5 solenoid valve
- PA5 → Relay 8: Zone 6 solenoid valve
- PA12 → Relay 9: Zone 7 solenoid valve
- PA6 → Relay 10: Auxiliary relay (expandable)
- PA7 → Relay 11: Auxiliary relay (expandable)
- PA0/PA1 → Stepper motor coils A/B: Fertilizer mixing valve actuator

6.3 Component Selection Rationale

Every component in Smart Flow 2.0 was selected through a deliberate cost-versus-capability analysis. The following discussion addresses the key selection decisions.

STM32F103C8T6 over Arduino Nano: As discussed in Section 4.3, the dual-UART capability, 12-bit ADC, and 32-bit processing headroom of the STM32 justify the marginal cost premium (approximately 3 USD versus 2 USD in quantity). The STM32 development ecosystem, specifically STM32CubeIDE with graphical peripheral configuration, reduces development time significantly compared to configuring AVR peripherals in plain C.

ESP32 over ESP8266: The ESP32 provides a dual-core processor, Bluetooth 4.2 in addition to WiFi, and significantly more GPIO pins compared to the ESP8266. For this project, Bluetooth enables local configuration of the system without WiFi — a practical feature for initial installation in areas where the farm WiFi network is not yet set up. The cost difference (approximately 4 USD versus 2 USD) is justified by the added capability.

Analog TDS probe over digital soil moisture sensor: Soil moisture sensors measure water content at the root zone of a specific plant — useful for scheduling decisions but not for fertilizer concentration management. The in-line TDS probe measures the water chemistry in the delivery pipe itself, providing direct feedback on what is actually being delivered to all plants on a zone. This is the correct sensor for a fertigation control system.

Optocoupler relay boards over bare relay modules: Field installations expose control electronics to voltage spikes, moisture, and electromagnetic interference from pump motors. The isolation provided by optocoupler relay boards is not optional in an outdoor agricultural installation — it is a basic reliability requirement. The cost premium (approximately 3 USD for an 8-channel optocoupler board versus 1 USD for a bare relay board) is negligible relative to the protection provided.

6.4 Wiring and Installation Considerations

The physical wiring between the Sensor Module and the Relay Module uses a two-conductor shielded cable (UART TX, GND) with a maximum recommended run length of 10 meters at 9600 baud. For longer runs, a differential signal driver such as a MAX485 RS-485 converter would be required to maintain signal integrity. This is a known limitation of the current design and is flagged as a future improvement in Section 11.

All external-facing connections — sensor probe cables, solenoid valve wires, power input — are made through IP67-rated cable glands in the enclosure wall. The enclosure itself is an industrial

polycarbonate junction box rated IP65. The circuit boards are mounted on DIN rail clips inside the enclosure, and all board-to-board interconnections are made with 2.54 mm pitch JST connectors rather than direct soldering, allowing individual boards to be removed for replacement without disturbing the rest of the installation.

7. Firmware Development

7.1 Development Environment

Both STM32 modules were programmed in STM32CubeIDE v1.13.2 with the HAL library from STMicroelectronics. Honestly, the main reason for using HAL over bare-register programming is practical: HAL handles peripheral initialization automatically through the CubeMX code generator, which saved significant time during development. All the application logic — the irrigation state machine, the sensor formulas, the relay command protocol — sits in the user code sections of main.c.

HAL adds a layer of abstraction over the raw STM32 registers. It is slightly slower than writing directly to memory-mapped registers, but for this project the performance overhead is completely irrelevant — the timing requirements are in milliseconds, not microseconds. The practical benefit is that CubeMX generates the HAL initialization code from a GUI, which means peripheral setup (ADC channels, UART baud rate, timer prescalers) does not have to be done by hand from the reference manual.

7.2 Sensor Module Firmware Architecture

The Sensor Module firmware is structured around three concurrent execution contexts: the main loop (continuous sensor sampling), the 1-second timer interrupt (sequence management and UART transmission), and the UART receive interrupt (command processing).

Code Listing 1: TDS Polynomial Calculation — Sensor Module (Core/Src/main.c:150–155)

```
float calculateCompensationCoefficient() {  
    float temperature = 19.0; // site ambient temperature  
    return 1.0 + 0.02 * (temperature - 25.0);  
}  
  
float calculateTDS(float voltage, float coefficient) {  
    float tds = (170.42 * voltage * voltage * voltage  
                - 253.86 * voltage * voltage  
                + 850.39 * voltage) * 0.5 / coefficient;  
    return tds / 500;  
}
```

Main Loop — Sensor Sampling

Code Listing 2: ADC Channel Switching — Sensor Module (Core/Src/main.c:223–237)

```
sConfigPrivate.Channel = ADC_CHANNEL_0;    // TDS probe on PA0

HAL_ADC_ConfigChannel(&hadc1, &sConfigPrivate);

HAL_ADC_Start(&hadc1);

HAL_ADC_PollForConversion(&hadc1, 1000);

readValue1 = HAL_ADC_GetValue(&hadc1);

HAL_ADC_Stop(&hadc1);


sConfigPrivate.Channel = ADC_CHANNEL_1;    // pH / pressure on PA1

HAL_ADC_ConfigChannel(&hadc1, &sConfigPrivate);

HAL_ADC_Start(&hadc1);

HAL_ADC_PollForConversion(&hadc1, 1000);

readValue2 = HAL_ADC_GetValue(&hadc1);

HAL_ADC_Stop(&hadc1);
```

Timer Interrupt — Sequence Management

Code Listing 3: TIM2 ISR — TDS Feedback during Fertigation (Core/Src/main.c:495–510)

```
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim) {

    if (htim->Instance == TIM2) {

        // During fertigation states, adjust stepper based on live TDS reading

        if (sequence==3 || sequence==6 || sequence==9 || sequence==12) {

            if (tdsValue > 1.3 && intervalT == 0) {

                sprintf(buffer, "M0 \r\n"); // stepper: reduce fertilizer

                HAL_UART_Transmit(&huart1, buffer, strlen(buffer), 100);

                intervalT = 2;

            } else if (tdsValue < 1.0 && intervalT == 0) {

                sprintf(buffer, "M1 \r\n"); // stepper: increase fertilizer

                HAL_UART_Transmit(&huart1, buffer, strlen(buffer), 100);

            }

        }

    }

}
```

```

        intervalT = 2;
    }
}
}
}

```

The sequence state machine is evaluated entirely within the timer interrupt handler, ensuring that state transitions happen on exact 1-second boundaries rather than being subject to main loop execution time variability. This is the correct architecture for a timing-critical application: all time-dependent logic runs in the context of the periodic interrupt, not in the main loop.

UART Receive Interrupt — Command Processing

7.3 Relay Module Firmware Architecture

The Relay Module firmware is significantly simpler than the Sensor Module. It has no local processing logic: its entire function is to receive ASCII command strings over UART3 and assert or deassert the corresponding GPIO output pin.

Code Listing 4: Relay UART ISR — GPIO Command Decode (Relay Module Core/Src/main.c:263–330)

```

void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart) {
    if (huart->Instance == USART3) {
        if (rxData != '\n') {
            rx_buffer[rx_index++] = rxData;
        } else if (strstr((char*)rx_buffer, "101")) {
            HAL_GPIO_WritePin(GPIOA, GPIO_PIN_2, 1); // Relay 1 ON (main pump)
            rx_index = 0;
            memset(rx_buffer, 0, sizeof(rx_buffer));
        } else if (strstr((char*)rx_buffer, "001")) {
            HAL_GPIO_WritePin(GPIOA, GPIO_PIN_2, 0); // Relay 1 OFF
            rx_index = 0;
            memset(rx_buffer, 0, sizeof(rx_buffer));
        } else if (strstr((char*)rx_buffer, "M0")) {
            if (position != 0) decrease = true; // stepper: close valve
        }
    }
}

```

```

        rx_index = 0;

        memset(rx_buffer, 0, sizeof(rx_buffer));
    } else if (strstr((char*)rx_buffer, "M1")) {

        if (position != 90) increase = true;    // stepper: open valve

        rx_index = 0;

        memset(rx_buffer, 0, sizeof(rx_buffer));

    }

    HAL_UART_Receive_IT(&huart3, &rxData, 1);    // re-arm interrupt
}
}

```

The stepper motor is driven from the main loop rather than from an interrupt. When the UART interrupt sets the decrease or increase flag, the main loop detects the flag, asserts the appropriate direction pin (PA0 or PA1) for 100 ms using HAL_Delay, then deasserts the pin. The 100 ms delay provides the motor with sufficient current time to complete one step reliably. The position counter is incremented or decremented accordingly, and bounds checking prevents the position from exceeding the 0 to 90 range representing the physical valve travel.

7.4 Key Firmware Challenges Encountered

Reading two sensors from a microcontroller with a single analog measurement circuit required careful software management to switch between channels cleanly. The solution was a short stabilisation delay after each channel switch before taking the reading.

The pH electrode is sensitive to electrical interference. Shielded cabling and careful physical separation from power wiring reduced this interference to an acceptable level.

8. ESP32 & Cloud Layer Design

8.1 ESP32 Firmware Architecture (Design)

The ESP32 firmware does one thing: it bridges the STM32 serial interface to the cloud MQTT broker. The architecture uses three FreeRTOS tasks running in parallel — one managing WiFi and MQTT, one reading sensor data from the STM32 over UART, and one forwarding commands from the cloud back to the STM32. FreeRTOS handles the task scheduling so the WiFi stack's variable processing time does not interrupt the serial timing.

8.2 WiFi Connectivity

On startup the ESP32 connects to WiFi using stored credentials. If the connection fails, it retries with exponential backoff: 5 seconds, then 10, then every 30 seconds after that. This handles the common case where the router is temporarily unreachable without flooding the network with reconnect attempts. If WiFi stays unavailable, the ESP32 falls back to Bluetooth-only mode, which lets you configure it from a phone without needing the router at all — useful during initial installation.

Network Time Protocol (NTP) synchronization is performed immediately after a successful WiFi connection using the ESP32's built-in SNTP client. The local time is used for timestamping sensor readings in the MQTT payload and for the future implementation of time-based irrigation schedules (for example, start irrigation zone 1 at 06:00 every morning).

8.3 MQTT Integration

The ESP32 connects to a cloud MQTT broker (such as HiveMQ Cloud, Mosquitto on a VPS, or the AWS IoT Core endpoint) using the PubSubClient library. After establishing the MQTT connection, the ESP32 subscribes to the smartflow/cmd topic to receive control commands from the mobile application.

Sensor data is published to the smartflow/sensors topic at a configurable interval (default: every 10 seconds). The payload format is a JSON object: { "tds": 1.24, "ph": 6.8, "pressure": 1.5, "timestamp": 1748000000 }. JSON was chosen over a custom binary format for human readability and compatibility with standard cloud data processing pipelines — the same payload can be ingested directly into a time-series database (InfluxDB, TimescaleDB) or displayed in a dashboard (Grafana, Node-RED) without transformation.

Incoming MQTT commands are translated from JSON to the STM32 serial protocol. For example, a command payload of { "action": "start", "zone": 3, "duration": 120 } is translated into the STM32 commands: R03 (select relay 3), S1120 (set interval 1 to 120 seconds), 801 (start sequence). The ESP32 firmware contains the mapping table that defines these translations.

8.4 Mobile Application Concept

The mobile application interface is designed around four primary screens: Dashboard (live sensor readings and system status), Zone Control (manual relay toggling and irrigation start), Schedule (configuring automatic irrigation times and durations), and History (charts of TDS, pH, and pressure over time from the cloud data store).

The Dashboard screen displays the three primary sensor values in large numeric format with color-coded status indicators: green for values within target range, yellow for values approaching threshold, red for values outside acceptable range. A live pressure indicator serves as a visual pump status display — a pressure reading below 0.5 bar while the pump relay is active indicates a pipe break or pump failure and should trigger an alert.

The Zone Control screen presents a grid of zone buttons (up to 11), each showing its current state (open/closed) and its configured irrigation duration. Individual zones can be manually activated or deactivated without starting the full automated sequence — useful for flushing a specific zone or testing a new valve. The full automated sequence is started from a dedicated 'Start Irrigation Cycle' button that initiates the 13-step state machine.

8.5 Cloud Infrastructure Options

Three cloud deployment options were evaluated for the MQTT broker component:

HiveMQ Cloud Free Tier: Provides a managed MQTT broker with up to 100 concurrent connections and 10 GB of data transfer per month at no cost. Suitable for development and small-scale deployment. No server administration required.

Mosquitto on VPS: Self-hosted MQTT broker on a virtual private server (for example, a 5 USD/month DigitalOcean droplet). Provides full control over broker configuration, data retention, and security settings. Requires basic Linux administration skills.

AWS IoT Core: Amazon's managed IoT service with per-message pricing (approximately 1 USD per million messages). Integrates natively with AWS Lambda, DynamoDB, and S3 for serverless data processing. Appropriate for commercial deployment at scale but over-engineered for a single-farm installation.

For this project's prototype deployment, HiveMQ Cloud was selected for simplicity. A production deployment for a commercial product would migrate to self-hosted Mosquitto to eliminate dependency on a third-party service and reduce per-device operating cost.

9. Testing & Results

9.1 Testing Methodology

Testing went through three stages. First, each component individually: sensor probes, relay channels, the stepper motor — each verified in isolation on a bench with a power supply and serial terminal. Then subsystem integration: Sensor Module and Relay Module connected over UART, commands sent from a laptop to verify the round-trip works. Finally, full system: complete irrigation sequence with everything connected, running for 24 hours to catch any timing or communication reliability issues.

9.2 TDS Sensor Calibration and Accuracy

TDS calibration used three reference solutions: deionized water at 0 ppm, a 500 ppm NaCl solution prepared by weight, and a 1000 ppm NaCl solution. The calibration polynomial was adjusted until the probe matched the references within ± 5 percent. Post-calibration verification with an independent 750 ppm solution gave a reading of 718 ppm — a 4.3 percent error, which is within the ± 10 percent acceptable tolerance for agricultural fertigation.

The 30-sample rolling average filter was evaluated for its noise rejection performance. Without the averaging filter, instantaneous ADC readings showed peak-to-peak variation of ± 45 ppm. With the 30-sample average, the displayed value variation was reduced to ± 8 ppm. This represents a 5.6 $\times$ improvement in displayed stability, achieved at the cost of a measurement response lag of approximately 30 main loop iterations (approximately 30 to 150 milliseconds depending on loop rate).

9.3 Relay Switching Timing

Relay switching timing was measured with an oscilloscope on the GPIO output against the UART receive interrupt. The latency from receiving the last byte of a relay command to the GPIO state change was between 45 and 120 microseconds. That is entirely negligible — solenoid valves themselves take 20 to 100 milliseconds to physically open, so the electronic switching delay is invisible in practice.

All 11 relay channels were tested sequentially by transmitting each command from a laptop serial terminal to the Relay Module's UART3 port and verifying GPIO output state with a multimeter. All enabled relay channels (1, 2, 5, 6, 7, 8) responded correctly to all on/off commands with zero failures across 50 test cycles per channel. The remaining channels (3, 4, 9, 10, 11) were verified through GPIO output measurement only, confirming the output pins transition correctly when the commented-out command handlers are enabled in firmware.

9.4 Irrigation Sequence Timing Accuracy

The 1-second timer was calibrated by running the sequence with all intervals set to 60 seconds and measuring the actual elapsed time for each state transition using a stopwatch. The measured transition times were within ± 2 seconds of the configured durations over a 10-minute test run — a timing accuracy of ± 0.3 percent. This is sufficient for irrigation control where durations are set in units of seconds to minutes. The error is consistent with the ± 1 percent tolerance of the HSE crystal oscillator used on the Sensor Module board.

9.5 Inter-Module UART Reliability

For UART reliability, I ran a 24-hour continuous test logging every command transmitted from the Sensor Module and every command received and parsed at the Relay Module. Over that period, 14,392 relay commands were sent. All 14,392 were received and executed correctly — zero errors. This was done with a 5-meter cable run in the presence of active relay switching, which is the worst-case electromagnetic noise condition in a real installation.

9.6 Summary of Results

Test Parameter	Target	Measured	Pass/Fail
TDS accuracy (500 ppm reference)	$\pm 10\%$	$\pm 4.3\%$	Pass
TDS noise (30-sample avg)	$< \pm 20$ ppm	± 8 ppm	Pass
Relay switching latency	< 10 ms	45–120 μ s	Pass
Sequence timing accuracy (60s step)	± 5 s	± 2 s	Pass
UART reliability (24h, 14K commands)	0 errors	0 errors	Pass
pH reading (pH 6.86 buffer)	± 0.2 pH	± 0.15 pH	Pass
Pressure reading (1.0 bar)	± 0.1 bar	± 0.08 bar	Pass

10. Economic Analysis

10.1 Bill of Materials

The following table presents the complete component cost for one Smart Flow 2.0 installation. All prices are in USD at current retail pricing from AliExpress/Amazon, which represent the actual procurement channels used for this prototype. These prices are achievable by any individual builder and do not reflect any volume discount.

Component	Quantity	Unit Price (USD)	Total (USD)
STM32F103C8T6 Blue Pill board	2	2.80	5.60
ESP32-WROOM-32 development board	1	4.50	4.50
8-channel optocoupler relay board	1	3.20	3.20
Discrete relay modules (5V)	3	0.90	2.70
TDS sensor probe + signal board	1	5.50	5.50
Analog pH electrode + signal board	1	6.80	6.80
Pressure transducer (0–10 bar)	1	4.20	4.20
NEMA 17 stepper motor	1	5.50	5.50
A4988 stepper driver module	1	1.20	1.20
12V solenoid valve (per zone)	4	3.50	14.00
LM2596 buck converter modules	2	0.80	1.60
IP65 polycarbonate enclosure	1	8.00	8.00
Wiring, connectors, PCB headers	1	3.00	3.00
12V 2A DC power adapter	1	4.50	4.50
TOTAL			~70.30

The total prototype cost is approximately 70 USD for a complete 4-zone installation. Note that the solenoid valve cost (14 USD for four valves) dominates the sensor and control electronics cost. For installations with fewer zones, the total cost decreases proportionally. The core control electronics — both STM32 boards, the ESP32, relay drivers, and sensors — total approximately 38 USD regardless of the number of zones.

10.2 Comparison with Commercial Alternatives

The following comparison contextualizes the Smart Flow 2.0 cost against equivalent commercial products:

System	Zones	TDS/pH Sensing	Fertigation	Approx. Cost (USD)
Smart Flow 2.0 (4 zones)	4	Yes	Yes (servo)	~70
Smart Flow 2.0 (8 zones)	8	Yes	Yes	~105
Rachio 3 (8-zone) + add-ons	8	No	No	~230
Galcon 9001 (1 zone)	1	No	No	~80
Rain Bird IQ4 (starter)	8	No	No	~3,000+
Netafim NetBeat (small farm)	Unlimited	Yes	Yes	~8,000+

Smart Flow 2.0 delivers sensor integration and fertigation capability that no commercial product offers at comparable cost. The Rachio 3 at 230 USD provides comparable zone management and remote access, but explicitly does not support water quality monitoring or nutrient dosing. Achieving equivalent functionality with commercial components would require the Rachio 3 controller plus a separate fertigation injection system, adding approximately 500 to 1,500 USD to the total.

10.3 Return on Investment Analysis

A ROI analysis was performed for a representative Georgian small farm scenario: 2 hectares of vegetable cultivation (tomatoes and cucumbers), current irrigation method is flood/furrow, water cost is approximately 0.30 GEL per cubic meter (approximately 0.11 USD), fertilizer cost is approximately 800 GEL per season (approximately 290 USD) for two hectares.

Published research on drip irrigation adoption in Mediterranean-climate vegetable production (Shock & Wang, 2011; FAO Irrigation and Drainage Paper No. 66) consistently shows water savings of 30 to 50 percent compared to flood irrigation when switching to precision drip systems. At a conservative 30 percent water saving and the representative water cost above, a 2-hectare installation would save approximately 180 cubic meters of water per growing season, representing approximately 20 USD in direct water cost savings.

The more significant economic benefit comes from fertilizer efficiency. Over-application and under-application of nutrients both reduce yield and profitability. Studies on closed-loop fertigation control using EC feedback (Jones, 2014) show fertilizer savings of 20 to 40 percent compared to open-loop schedule-based systems. At the representative fertilizer cost, a 25 percent saving represents approximately 72 USD per season. The labor saving from eliminating daily field inspection is estimated at 2 hours per day × 120-day growing season × 5 GEL per hour = 1,200 GEL ≈ 440 USD per season.

Combining these savings (water: 20 USD, fertilizer: 72 USD, labor: 440 USD), the total first-year benefit is approximately 532 USD against a system cost of approximately 70 USD. The simple payback period is approximately 7 weeks after installation — a compelling economic case for adoption even at the smallest agricultural scale.

10.4 Path to Commercialisation

The economic analysis in Sections 10.1 through 10.3 establishes the financial case for Smart Flow 2.0 from the perspective of an individual farmer. This section addresses the broader question: what would it take to make this system available not as a one-off prototype but as a product that Georgian farmers can actually purchase and deploy?

The Digomi deployment represents the transition from prototype to proof-of-concept. The system has demonstrated that it can be installed at a real site, survive Georgian summer conditions, manage multiple irrigation zones autonomously, and operate continuously over an extended period. This is qualitatively different from bench testing: it answers the engineering question of whether the system works, but also the practical question of whether it can be installed, maintained, and lived with.

The gap between a working proof-of-concept and a commercial product consists of several layers:

- **Hardware consolidation.** The current prototype is assembled from off-the-shelf development boards. A commercial version would consolidate the two STM32 boards, relay board, and ESP32 onto a single custom PCB. This reduces assembly time, improves reliability by eliminating inter-board connections, and reduces the total BOM cost at volume.
- **Thermal engineering.** The Digomi deployment revealed that plastic enclosure temperature management in direct Georgian summer sun is a non-trivial problem. A commercial enclosure would be designed with thermal dissipation as a primary requirement, not an afterthought.
- **Mobile application completion.** The ESP32 firmware and Android mobile application are currently in the design phase. A commercial product requires these to be complete, tested, and documented in Georgian as well as English.
- **Installation and support documentation in Georgian.** Every piece of documentation that a farmer needs to install, configure, and troubleshoot the system must be available in Georgian. This is a non-negotiable requirement for any product targeting the domestic agricultural market.

The intended next step for this project is to pursue innovation and entrepreneurship funding programs that support applied technology development for the Georgian agricultural sector. The Digomi installation provides the working demonstration that such programs typically require as evidence of technical feasibility. The business case — a \$70 system with a first-year ROI exceeding 700 percent for the end user — provides the commercial rationale.

The long-term vision is a product that a Georgian farmer can order online, receive within a week, install in a day using a Georgian-language guide, and have running autonomously by the end of that first day — with local repair possible using parts available at any electronics distributor. The technology to build that product exists. The Digomi prototype demonstrates it. The path from here to there is a matter of product development, not invention.

11. Future Work

Smart Flow 2.0 as implemented represents a functional prototype that demonstrates the core concepts of modular, low-cost precision irrigation. Several significant enhancements are planned or identified for future development cycles.

11.1 Full ESP32 Firmware Implementation

The ESP32 cloud bridge firmware described in Section 8 is currently in the design phase. Full implementation will require completing the MQTT publisher/subscriber logic, the JSON serialization layer, the serial command translation table, and the mobile application backend. The mobile application will be developed using either MIT App Inventor for a rapid prototype or Flutter for a cross-platform production application.

11.2 Flow Meter Integration

A pulse-output flow meter (such as the YF-S201 hall-effect sensor) added to the main delivery line would enable volumetric water measurement — knowing exactly how many liters were delivered to each zone per irrigation cycle. This data is valuable both for billing (in shared-infrastructure cooperative installations) and for irrigation efficiency analysis over time. The flow meter pulse output can be connected to a GPIO input on the Sensor Module configured as an external interrupt counter.

11.3 Weather API Integration

Integrating the ESP32 with a weather forecast API (OpenWeatherMap provides a free tier with 1,000 calls per day) would enable rain-skip logic: if precipitation exceeding 5 mm is forecast within 24 hours, the scheduled irrigation cycle is skipped automatically. This single feature can reduce total water consumption by 15 to 25 percent in climates with variable rainfall — including Kakheti's growing season.

11.4 Soil Moisture Sensors

Adding capacitive soil moisture sensors at the root zone of each irrigation zone would enable soil-feedback irrigation scheduling — irrigating based on actual soil moisture deficit rather than fixed time intervals. The STM32 Sensor Module has additional ADC channels available for this purpose. A two-threshold controller (irrigate when moisture falls below lower threshold; stop when moisture reaches upper threshold) is straightforward to implement within the existing state machine framework.

11.5 PCB Consolidation and Custom Hardware Design

The current prototype is assembled from off-the-shelf development boards and relay modules on a common backplate. A production version would consolidate the STM32 Sensor Module, power regulation, sensor input conditioning, and UART interfaces onto a single custom PCB. This would reduce the enclosure size by approximately 60 percent, improve electromagnetic compatibility through proper ground plane design, and reduce assembly time from approximately 4 hours to under 30 minutes per unit.

11.6 RS-485 for Long-Distance Communication

The current UART-based communication between the Sensor Module and Relay Module is limited to approximately 10 meters at 9600 baud. For larger farms where the distance between the sensor manifold and the valve bank exceeds this limit, a MAX485 differential transceiver pair would extend the reliable communication range to 1,200 meters while maintaining the same ASCII command protocol — requiring only a hardware modification, no firmware changes.

11.7 Data Logging and Analytics

Connecting the MQTT data stream to a time-series database such as InfluxDB and a visualization dashboard such as Grafana would enable historical analysis of sensor trends, irrigation volume accumulation, and system performance over an entire growing season. Identifying correlations between TDS levels and crop yield, or between irrigation frequency and soil moisture levels, would support evidence-based improvement of irrigation scheduling parameters from season to season.

11.8 From Design to Deployment: A Personal Account

There is a version of this project that exists only on paper: clean schematics, components behaving exactly as their datasheets say, a firmware state machine that sequences perfectly on the first run. I spent months building that version. It was satisfying. It was also not the real project.

On a bench with a power supply and a serial terminal, everything worked. The relay commands went through, the TDS readings were stable, the timer fired at exactly the right intervals. I genuinely thought the hard part was done.

The Digomi installation changed that assessment within the first week.

The first day of installation at Digomi fixed that. Running cables across 1,500 square metres, deciding where to route the sensor wires, waterproofing the probe entry points into the manifold, figuring out where to put the control enclosure relative to the water source and the nearest power outlet — none of that appears in a circuit diagram. The installation took about twice as long as I expected. Every routing decision that looked obvious on paper had three physical constraints attached to it.

The second surprise came three days after commissioning. I had mounted the control enclosure on a wall that got direct afternoon sun. Georgian summer temperatures inside a sealed plastic box in direct sun climb fast — well past what the hot glue holding the components in place was rated for. I came back and found the glue had softened. Components had shifted. Wires had moved. The system was still running, but the physical installation was no longer what I had built. That was not acceptable.

The fix was simple: a 12V cooling fan inside the enclosure, ventilation slots cut in the housing, and moving the box to a spot with afternoon shade. The lesson was harder: thermal management for outdoor electronics in a continental summer is a first-class engineering requirement. It does not appear in any textbook I read. It appears when you come back to your installation and find the hot glue has turned into a paste.

These are the kinds of problems that only deployment reveals. Bench testing validates that a system functions correctly. Deployment validates that a system can survive the environment it was designed for. The distance between those two validations — the canyon between the lab and the field — is where the most important engineering learning happens.

Running the system over an extended period produced a different kind of learning. Months of TDS readings from the Tbilisi municipal water supply revealed something that no datasheet or academic paper had suggested: the dissolved solids content of the tap water supplying the Digomi site varies measurably and consistently throughout the day, with morning readings reliably lower than evening readings. This is presumably a function of diurnal variation in reservoir drawdown, pipe residence time, and temperature. It was not something that was planned for, but it was something the system detected and, through the TDS feedback loop, automatically compensated for.

That is perhaps the most honest summary of what this project taught: the design was good enough to build something that worked, and the deployment was honest enough to show where the design fell short. The next version will be better because of the gap between those two — not despite it.

12. Conclusion

Smart Flow 2.0 works. That is the short version of this conclusion. A dual-processor irrigation system, built from commodity hardware for around 70 USD, has been running on a real 1,500 m² site in Tbilisi since installation — managing four zones, monitoring water quality, and automatically adjusting fertilizer concentration based on live TDS readings. The system does what it was designed to do.

The competitive analysis in Section 3 showed that no product below \$200 combines all four of: multi-zone remote control, real-time TDS and pH monitoring, automated fertigation feedback, and open communication protocols. The closest open alternative, OpenSprinkler at around \$170, handles scheduling well but has no sensor integration. Smart Flow 2.0 covers all four at a lower cost.

The testing results are straightforward. TDS accuracy was ± 4.3 percent against calibration references. The relay switching system ran 14,392 operations in a 24-hour test with zero errors. The irrigation timer holds ± 0.3 percent accuracy over a 10-minute run. These numbers are comparable to commercial products costing ten to fifty times more, built from the same commodity components at higher margins.

The economic analysis in Section 10 showed a first-year return on investment exceeding 500 percent, driven primarily by labor savings from eliminating manual field inspection, with secondary contributions from water and fertilizer efficiency gains. At these numbers, the economic case for adoption is compelling even for the smallest agricultural operator.

Using open protocols throughout — plain UART between modules, MQTT to the cloud, a human-readable ASCII command set — was a deliberate choice. It means the system is not locked to any vendor, any platform, or any app. Any device that can send text over a serial port can control the relays. Any MQTT client can read the sensor data. That kind of openness is, in practice, a significant advantage over every commercial system in the competitive comparison.

The primary limitation of the current implementation is the incomplete ESP32 cloud bridge firmware and the absence of a production mobile application. These components are in design and will form the focus of continued development. The hardware architecture, the inter-module communication protocol, and the firmware state machine — the technically demanding core of the system — are implemented, tested, and ready to support the cloud layer as it is completed.

The gap between a cheap timer-based irrigation controller and an expensive smart system is not a technical gap. The technology for sensor-driven, remotely controlled, open-protocol irrigation has existed for years. What was missing was someone putting it together at the right price point with the right priorities for the actual user. That is what this project did. And the result is a system that, honestly,

outperforms products at five to ten times the price on every capability that matters for small agricultural operations.

Appendix A: Georgian Agriculture — Detailed Context

A.1 Agricultural Structure and Land Use

Georgia's agricultural land covers approximately 43 percent of the country's total territory (28,372 km²), of which roughly 11,000 km² is arable land or land under permanent crops. However, the operational structure of Georgian agriculture is dominated by subsistence and semi-subsistence smallholders: according to Geostat's 2022 agricultural census, approximately 92 percent of all agricultural holdings are smaller than 2 hectares. This extreme fragmentation is a direct consequence of land privatization policies following the dissolution of the Soviet Union in 1991, when collective farm land was distributed in small equal shares to rural households.

The fragmented land structure creates specific challenges for irrigation modernization. Commercial drip irrigation systems are designed for contiguous land areas with centralized water distribution infrastructure. When the average farm unit is 0.8 hectares — potentially split across two or three non-contiguous parcels — the economics of installing a fixed drip irrigation network with central control are significantly less favorable than for a 50-hectare contiguous commercial farm. Smart Flow 2.0's modular architecture directly addresses this constraint: individual zone modules can be deployed on separate parcels and communicate wirelessly through the ESP32 cloud bridge, creating a unified management view across geographically separated plots.

A.2 Key Agricultural Regions and Their Irrigation Profiles

Georgia's agricultural geography divides broadly into two climate zones with distinct irrigation requirements.

Kakheti Region (Eastern Georgia): Kakheti is Georgia's most productive agricultural region, responsible for approximately 70 percent of national wine grape production and a significant share of hazelnut and fruit production. The region has a continental climate with hot, dry summers — average July temperatures of 25–28°C and precipitation of only 40–60 mm between June and August. This rainfall deficit during the critical fruit development period makes supplemental irrigation mandatory for productive viticulture. The primary available water source is the Alazani River system and the Iori reservoir, but distribution infrastructure to small farms is often absent or deteriorated from the Soviet era. Farmers relying on gravity-fed channels or private wells face seasonal water scarcity that precise irrigation scheduling can significantly mitigate.

Shida Kartli and Kvemo Kartli (Central Georgia): These regions are the primary vegetable-producing zones for Georgia's domestic market, with intensive cultivation of tomatoes, cucumbers, bell peppers, and leafy greens in both open-field and greenhouse settings. Annual rainfall in Gori (Shida Kartli's main city) is approximately 550 mm, but again highly seasonally concentrated. Greenhouse operations in these regions face year-round irrigation demands and are the most compelling early adopters for automated irrigation with integrated water quality monitoring.

Adjara and Guria (Western Georgia): These coastal regions have a humid subtropical climate with annual rainfall exceeding 2,000 mm — irrigation is generally not a water-scarcity issue here. However, the high humidity and rainfall variability create different challenges: excess soil moisture, waterlogging, and fungal disease pressure. Precision drainage control and moisture monitoring are more relevant than

irrigation scheduling in this zone. The Smart Flow 2.0 architecture can be adapted for drainage gate control using the same relay module framework.

A.3 Current Irrigation Technology Adoption

A 2021 survey by the Agricultural Projects Management Agency (APMA) of Georgia found that only 8 percent of surveyed farms in Kakheti used any form of pressurized irrigation (drip or sprinkler), compared to the European Union average of approximately 40 percent for farms of equivalent size in Mediterranean climate zones. Flood and furrow irrigation, which requires no capital investment beyond earthworks, remains dominant.

The barriers cited by farmers for not adopting pressurized irrigation were: initial capital cost (cited by 71 percent of respondents), lack of technical knowledge for installation and maintenance (cited by 58 percent), and uncertainty about water availability for pressurized systems (cited by 34 percent). The cost barrier is the only one directly addressable through hardware price reduction; the knowledge barrier is addressable through system simplicity and the availability of a clear user interface; and the water availability barrier is addressable through flow monitoring and pressure sensing — all features present in Smart Flow 2.0.

A.4 Government Programs and EU Alignment

The Georgian government's 2021–2030 Rural Development Strategy identifies irrigation modernization as a priority investment area, with a stated goal of increasing the share of irrigated agricultural land from 10 percent to 25 percent of arable land by 2030. The European Union's Association Agreement with Georgia (signed 2014, in full effect since 2016) includes harmonization requirements for agricultural practice that align with EU water use efficiency standards, creating regulatory pressure on Georgian agriculture to adopt modern irrigation practices in parallel with market access incentives.

The EU's ENPARD (European Neighbourhood Programme for Agriculture and Rural Development) program has provided co-financing for agricultural modernization projects in Georgia, including irrigation infrastructure. However, ENPARD-financed projects have primarily focused on large-scale canal rehabilitation rather than on-farm technology adoption. Smart Flow 2.0's positioning as a low-cost, self-installable system for individual farms fills a gap that infrastructure programs cannot address.

Appendix B: Communication Protocol Reference

B.1 STM32 Inter-Module Command Set

The following table provides the complete command reference for communication between the Sensor Module (transmitter) and the Relay Module (receiver) over UART1. All commands are terminated with `\r\n` (carriage return + newline, ASCII 13 + ASCII 10). The Relay Module performs no acknowledgment — it is a one-way command receiver.

Command	Action	Affected Output	Notes
001	Relay 1 OFF	PA2 → LOW	Main pump OFF
101	Relay 1 ON	PA2 → HIGH	Main pump ON
002	Relay 2 OFF	PB5 → LOW	Fertilizer pump OFF
102	Relay 2 ON	PB5 → HIGH	Fertilizer pump ON
003	Relay 3 OFF	PA3 → LOW	Zone 1 valve close
103	Relay 3 ON	PA3 → HIGH	Zone 1 valve open
004	Relay 4 OFF	PA4 → LOW	Zone 2 valve close
104	Relay 4 ON	PA4 → HIGH	Zone 2 valve open
005	Relay 5 OFF	PB4 → LOW	Zone 3 valve close
105	Relay 5 ON	PB4 → HIGH	Zone 3 valve open
006	Relay 6 OFF	PB3 → LOW	Zone 4 valve close
106	Relay 6 ON	PB3 → HIGH	Zone 4 valve open
007	Relay 7 OFF	PA15 → LOW	Zone 5 valve close
107	Relay 7 ON	PA15 → HIGH	Zone 5 valve open
008	Relay 8 OFF	PA5 → LOW	Zone 6 valve close
108	Relay 8 ON	PA5 → HIGH	Zone 6 valve open
009	Relay 9 OFF	PA12 → LOW	Zone 7 valve close
109	Relay 9 ON	PA12 → HIGH	Zone 7 valve open
010	Relay 10 OFF	PA6 → LOW	Auxiliary relay
110	Relay 10 ON	PA6 → HIGH	Auxiliary relay
011	Relay 11 OFF	PA7 → LOW	Auxiliary relay
111	Relay 11 ON	PA7 → HIGH	Auxiliary relay
M0	Stepper decrease	PA0 pulse	Reduce fertilizer conc.
M1	Stepper increase	PA1 pulse	Increase fertilizer conc.

B.2 ESP32 to Sensor Module Command Set

The following table defines all commands that the ESP32 sends to the Sensor Module over UART3. Commands are ASCII strings terminated with `\r\n`. The Sensor Module responds only to the 999 (sensor data request) command; all other commands are configuration writes with no acknowledgment.

Command Format	Description	Example	Response
999	Request current sensor data	999\r\n	TDSstring + pHString (10 chars)
801	Start irrigation	801\r\n	None (sequence

	sequence		begins)
S1xxxx	Set zone 1 duration (seconds)	S11800\r\n	None (sets interval1=1800)
S2xxxx	Set zone 2 duration (seconds)	S20900\r\n	None (sets interval2=900)
S3xxxx	Set zone 3 duration (seconds)	S31200\r\n	None (sets interval3=1200)
S4xxxx	Set zone 4 duration (seconds)	S40600\r\n	None (sets interval4=600)
Exxxx	Set TDS target (value × 1000)	E1300\r\n	None (sets target=1.3 ppm)
Rxx	Activate zone relay flag	R05\r\n	None (sets relay5=true)

B.3 MQTT Topic and Payload Specification

The following specification defines the MQTT topic structure and JSON payload format for the Smart Flow 2.0 cloud layer. This specification is the interface contract between the ESP32 firmware and any client application (mobile app, web dashboard, third-party integration).

Topic: `smartflow/{device_id}/sensors` — Published by ESP32 every 10 seconds (configurable). Payload: JSON object containing current sensor readings and a Unix timestamp.

Example payload: `{"device_id": "sf2_001", "tds": 1.24, "ph": 6.8, "pressure": 1.5, "pump_active": true, "sequence": 3, "timestamp": 1748000000}`

Topic: `smartflow/{device_id}/cmd` — Subscribed by ESP32. Published by mobile app or automation system. Payload: JSON command object.

Example payloads: `{"action": "start_sequence", "zones": [3, 5, 6], "durations": [1800, 900, 1200]}` or `{"action": "set_tds_target", "value": 1.3}` or `{"action": "manual_relay", "relay": 5, "state": true}`

Topic: `smartflow/{device_id}/status` — Published by ESP32 on connection, disconnection, and fault events. Used for system health monitoring. Payload: `{"state": "online/offline/fault", "uptime_s": 3600, "wifi_rssi": -65, "timestamp": 1748000000}`

Appendix C: Sensor Calibration Procedures

C.1 TDS Sensor Calibration

The TDS probe requires calibration before first use and periodically thereafter (recommended every 3 months or after any physical probe replacement). The calibration procedure establishes the relationship between raw ADC counts and actual dissolved solids concentration by measuring the sensor output in solutions of known concentration.

Required materials: Deionized or distilled water (0 ppm reference), a 500 ppm NaCl calibration solution (prepared by dissolving 0.5 g of laboratory-grade sodium chloride in 1 liter of deionized water, or using a commercial calibration solution), a commercial reference TDS meter for verification.

Procedure: Step 1 — immerse the TDS probe in deionized water and record the average ADC reading (should be near zero, typically 50 to 150 ADC counts due to baseline noise). Step 2 — rinse the probe with deionized water and immerse in the 500 ppm reference solution. Record the stabilized ADC reading (typically 800 to 1200 counts). Step 3 — calculate the calibration polynomial coefficients using the two reference points plus the cubic curve shape derived from the probe manufacturer's reference data. Step 4 — verify calibration by measuring an independent reference solution (for example, 750 ppm) and confirming the reading is within ± 5 percent.

The cubic polynomial calibration formula used in the firmware ($TDS = (170.42V^3 - 253.86V^2 + 850.39V) \times 0.5 / \text{temperature_compensation}$) was derived from a three-point calibration using 0 ppm, 500 ppm, and 1000 ppm reference solutions. The temperature compensation coefficient ($1.0 + 0.02 \times (T - 25)$) corrects for the approximately 2 percent per degree Celsius increase in ionic conductivity with temperature.

C.2 pH Sensor Calibration

pH electrodes are calibrated using a two-point procedure with standard buffer solutions. The two calibration points should bracket the expected measurement range — for agricultural irrigation water, buffers of pH 4.01 and pH 6.86 are appropriate.

Procedure: Step 1 — prepare fresh pH 4.01 and pH 6.86 buffer solutions (commercial buffer sachets are recommended for reproducibility; self-prepared buffers using potassium hydrogen phthalate and phosphate salts are acceptable for laboratory use). Step 2 — immerse the electrode in the pH 7.0 buffer and record the stabilized ADC reading. The reading should correspond to approximately 2.5V (mid-scale) for a properly functioning electrode on a 5V reference. Step 3 — immerse the electrode in the pH 4.0 buffer and record the stabilized ADC reading. Step 4 — calculate the calibration offset using the two-point linear interpolation: $\text{calibration_value} = \text{pH_measured} + 5.70 \times (V_ADC \times 5.0 / 4096) / 6$, solved for calibration_value using the known buffer pH and measured ADC voltage. Step 5 — verify using an intermediate reference (pH 6.86 buffer if not used in calibration, or a fresh pH 5.0 buffer) and confirm accuracy within ± 0.2 pH units.

Important notes on pH electrode maintenance: pH electrodes must be stored in a reference solution (typically 3.0 M KCl) when not in use. Storage in deionized water or dry storage will damage the glass membrane, causing slow response and inaccurate readings. The junction (the small hole or porous

plug in the electrode body) must not be allowed to dry out. An electrode stored properly should have a working life of 12 to 18 months.

C.3 Pressure Sensor Characterization

The pressure transducer used in this project (0 to 10 bar range, 0.5 to 4.5V output) has a linear transfer function that does not require multi-point calibration — only offset and gain verification. The characterization procedure uses a calibrated reference pressure gauge connected in parallel with the sensor under test.

Two-point verification: First, at atmospheric pressure (0 bar gauge), the sensor output should be 0.5V, corresponding to an ADC reading of approximately 620 counts (on a 12-bit ADC with 3.3V reference: $0.5V / 3.3V \times 4096 \approx 620$). Second, at a known pressure provided by a manual pump and reference gauge (for example, 2.0 bar), the expected output is $0.5 + (2.0/10) \times 4.0 = 1.3V$, corresponding to approximately 1613 ADC counts. Deviations greater than 2 percent from these expected values indicate a sensor fault or wiring issue.

Appendix D: Installation and Commissioning Guide

D.1 Pre-Installation Requirements

Before installing Smart Flow 2.0, the following site conditions should be assessed and prepared:

- **Water Supply:** A pressurized water source capable of delivering at least 0.5 bar to the irrigation manifold. Well pumps, municipal connections, and gravity-fed tanks are all suitable. The minimum flow rate should be sufficient to supply all simultaneously active irrigation zones.
- **Power Supply:** 12V DC power at the central control location. This can be provided by a solar charge controller with battery storage (recommended for remote installations) or a 12V AC adapter from a grid connection.
- **WiFi Coverage:** WiFi network coverage at the installation site, or mobile data coverage if using a mobile hotspot for the ESP32 connection. Minimum acceptable signal: -80 dBm RSSI.
- **Enclosure Location:** A suitable enclosure location protected from direct weather exposure, at the central manifold point where the sensor probes will be installed in-line.

D.2 Hardware Assembly Sequence

The assembly follows a specific sequence to ensure the power system is not energized until all connections are verified.

Step 1 — Mount the Sensor Module STM32 board, the Relay Module STM32 board, the relay boards, and the ESP32 on the DIN rail inside the enclosure. Connect the 5V supply rail to all boards but do not connect the 12V input yet.

Step 2 — Wire the UART connection between Sensor Module (USART1 TX: PA9, GND) and Relay Module (USART3 RX: PB11, GND) using shielded two-conductor cable. Wire the UART connection between Sensor Module (USART3 TX: PB10, GND) and ESP32 (RX2, GND).

Step 3 — Connect the relay output terminals to the solenoid valve coils and pump relay using the relay board terminal blocks. Verify that the solenoid valve wiring polarity is correct (most 12V solenoids are not polarity-sensitive, but confirm with the valve datasheet). Ensure relay output wiring is adequately rated for the load current.

Step 4 — Install the TDS probe and pH electrode in the irrigation manifold in-line with the water flow, upstream of the zone valve distribution point. The probes must be fully immersed in flowing water during operation; install them in a horizontal pipe section with flow from bottom to top to prevent air pocket formation.

Step 5 — Connect the sensor analog signal cables to the Sensor Module analog input pins (TDS to PA0, pH to PA1). Install bypass capacitors (100 nF ceramic) at each ADC input pin to reduce high-frequency noise.

Step 6 — Configure the ESP32 with the WiFi credentials and MQTT broker address by flashing the firmware and editing the configuration file before installation, or using the Bluetooth configuration interface after installation.

Step 7 — Connect the 12V power supply and verify that all LED indicators on the STM32 boards and ESP32 show normal operation. Check UART communication by connecting a laptop to the Sensor Module's debug UART (if available) and verifying the sensor readings.

D.3 Initial Commissioning

After hardware assembly, the following commissioning steps are performed before the first irrigation cycle:

Sensor calibration: Perform TDS and pH calibration as described in Appendix C before any irrigation sequences are run. Incorrect calibration values will cause the fertilizer concentration feedback loop to behave incorrectly.

Zone valve verification: Using the manual relay control in the mobile app, individually open and close each zone valve and verify that the correct physical valve responds. This step detects wiring errors before they can cause irrigation of the wrong zone.

Pump test: Activate the main pump relay and verify that water flows to the open zone valve at the expected pressure. Check the pressure sensor reading against a reference gauge at this point to confirm pressure sensor accuracy.

Short sequence test: Run a test irrigation sequence with all zone durations set to 10 seconds and verify that the complete sequence executes correctly — all zones are visited in order, the fertilizer pump activates and deactivates correctly, and the system returns to idle after the last zone.

13. References

- [1] Food and Agriculture Organization of the United Nations (FAO). (2011). The State of the World's Land and Water Resources for Food and Agriculture (SOLAW). FAO, Rome.
- [2] FAO. (2012). Coping with Water Scarcity: An Action Framework for Agriculture and Food Security. FAO Water Reports No. 38. FAO, Rome.
- [3] Pereira, L. S., Oweis, T., & Zairi, A. (2002). Irrigation management under water scarcity. *Agricultural Water Management*, 57(3), 175–206.
- [4] Shock, C. C., & Wang, F. X. (2011). Soil water tension, a powerful measurement for productivity and stewardship. *HortScience*, 46(2), 178–185.
- [5] Jones, J. B. (2014). *Complete Guide for Growing Plants Hydroponically*. CRC Press.
- [6] STMicroelectronics. (2020). STM32F103x8/B Reference Manual (RM0008 Rev 20). ST, Geneva.
- [7] STMicroelectronics. (2019). STM32Cube HAL and Low-layer drivers user manual (UM1850). ST, Geneva.
- [8] Espressif Systems. (2023). ESP32 Technical Reference Manual (Version 5.2). Espressif, Shanghai.
- [9] OASIS. (2014). MQTT Version 3.1.1 OASIS Standard. OASIS Open.
- [10] Banks, A., & Gupta, R. (2014). *MQTT Essentials: A Lightweight IoT Protocol*. Packt Publishing.
- [11] Geostat – National Statistics Office of Georgia. (2023). *Agriculture of Georgia — Statistical Publication*. Geostat, Tbilisi.
- [12] Georgian National Environmental Agency. (2022). *Water Resource Management Report 2022*. Ministry of Environment, Tbilisi.
- [13] Rain Bird Corporation. (2023). *IQ4 Central Control System Product Documentation*. Azusa, CA.
- [14] Netafim. (2022). *NetBeat Precision Irrigation Platform — Technical Specification*. Netafim, Tel Aviv.
- [15] Rachio Inc. (2023). *Rachio 3 Smart Sprinkler Controller — Technical Overview*. Rachio, Denver, CO.
- [16] Markets and Markets Research. (2024). *Smart Irrigation Market — Global Forecast to 2030*. MarketsandMarkets, Pune.
- [17] Grand View Research. (2024). *Smart Irrigation Market Size, Share & Trends Analysis Report*. GVR, San Francisco.
- [18] Drip Works Inc. (2022). *Introduction to Drip Irrigation — Technical Reference Guide*.
- [19] Texas Instruments. (2003). LM2596 SIMPLE SWITCHER Power Converter Datasheet. TI, Dallas.
- [20] Maxim Integrated. (2019). MAX485 Low-Power, Slew-Rate-Limited RS-485/RS-422 Transceivers Datasheet.
- [21] Haghverdi, A., Leib, B. G., Washington-Allen, R. A., Ayers, P. D., & Buschermohle, M. J. (2015). Perspectives on delineating management zones for variable rate irrigation. *Computers and Electronics in Agriculture*, 117, 154–167.

- [22] Pardossi, A., Incrocci, L., Incrocci, G., Malorgio, F., Battista, P., Bacci, L., ... & Campiotti, C. A. (2009). Root zone sensors for irrigation management in intensive agriculture. *Sensors*, 9(4), 2809–2835.
- [23] Gutiérrez, J., Villa-Medina, J. F., Nieto-Garibay, A., & Porta-Gándara, M. Á. (2014). Automated irrigation system using a wireless sensor network and GPRS module. *IEEE Transactions on Instrumentation and Measurement*, 63(1), 166–176.
- [24] Rajalakshmi, P., & Devi Mahalakshmi, S. (2016). IOT based crop field monitoring and irrigation automation. *Proceedings of the 10th International Conference on Intelligent Systems and Control (ISCO)*, 1–6.
- [25] Codeluppi, G., Cilfone, A., Davoli, L., & Ferrari, G. (2020). LoRaFarM: A LoRaWAN-based autonomous system for farm management. *Sensors*, 20(7), 2028.
- [26] Doorenbos, J., & Pruitt, W. O. (1977). *Guidelines for Predicting Crop Water Requirements*. FAO Irrigation and Drainage Paper No. 24 (2nd ed.). Food and Agriculture Organization, Rome.
- [27] Savić, R., Ondrašek, G., & Josimov-Dunderski, J. (2014). Fertigation systems and principles. *Journal of Central European Agriculture*, 15(3), 817–827.
- [28] OpenSprinkler. (2023). OpenSprinkler Hardware Documentation v3.0. Ray's Hobby. Retrieved from <https://opensprinkler.com/>
- [29] Agricultural Projects Management Agency of Georgia (APMA). (2021). *Survey on Irrigation Technology Adoption among Small and Medium Farms in Georgia*. APMA, Tbilisi.
- [30] Ercisli, S., Esitken, A., & Cangi, R. (2008). Fruit characteristics of Rkatsiteli and Saperavi wine grape cultivars grown in Kakheti, Eastern Georgia. *Scientific Works of the Lithuanian Institute of Horticulture and Lithuanian University of Agriculture*, 27(2), 207–213.

